

Agentic AI, in *production*

Ten engineering posts on building, running and paying for AI agents, digested for reading without a connection. Each digest is an original summary with the link to the full post.

PREPARED 29 SEPTEMBER 2026

FROM DRUPESH2008.GITHUB.IO/TECH-BLOGS

SOURCES STRIPE · UBER · RAMP · ANTHROPIC · MANUS · COGNITION ·
CLOUDFLARE · AIRBNB · LINKEDIN

CONTENTS

Ten posts, three threads

The posts are grouped by the question they answer. If you only have one flight, read the middle thread first: it explains the vocabulary the other two use.

01 Minions: Stripe's one-shot, end-to-end coding agents

STRIPE · CODING AGENTS AT COMPANY SCALE

02 Running a Software Factory Efficiently at Uber Scale

UBER · CODING AGENTS AT COMPANY SCALE

03 Integrations That Write Themselves

RAMP · CODING AGENTS AT COMPANY SCALE

04 How we built our multi-agent research system

ANTHROPIC · HOW TO BUILD AN AGENT THAT HOLDS UP

05 Context Engineering for AI Agents: Lessons from Building Manus

MANUS · HOW TO BUILD AN AGENT THAT HOLDS UP

06 Multi-Agents: What's Actually Working

COGNITION · HOW TO BUILD AN AGENT THAT HOLDS UP

07 Code Mode: the better way to use MCP

CLOUDFLARE · HOW TO BUILD AN AGENT THAT HOLDS UP

08 QueryGPT: Natural Language to SQL Using Generative AI

UBER · AGENTS INSIDE THE PRODUCT

09 Automation Platform v2: Improving conversational AI at Airbnb

AIRBNB · AGENTS INSIDE THE PRODUCT

10 How we engineered LinkedIn's Hiring Assistant

LINKEDIN · AGENTS INSIDE THE PRODUCT

READING ORDER

Three threads

I. Coding agents at company scale

Three companies that let agents write production code unattended, and what they had to build underneath so that the output could be trusted at volume. Stripe (01) is the fullest account; Uber (02) is about the cost and governance that follow; Ramp (03) is the same idea applied to a single, well-shaped class of work.

II. How to build an agent that holds up

Four posts from teams that ship agents for a living: when to parallelise (04, 06), how to shape context (05), what tool calling should look like (07), and why the model is rarely the bottleneck. Start here if you only have one flight. The vocabulary in these four (orchestrator, subagent, context window, tool definition, sandbox) is used without explanation everywhere else.

III. Agents inside the product

Three customer-facing systems: text-to-SQL for operations teams (08), LLM-driven customer support (09), and a recruiting agent (10). The common thread is decomposition, guardrails and cheap human checkpoints, and each post is honest about the version that did not work first.

A note on the numbers

Several posts report adoption figures, pull-request counts or hours saved. The digests repeat them as those companies' own claims, not as independent measurements, and round them where the original was a precise figure that has probably moved since.

01

STRIPE · FINTECH

Minions: Stripe's one-shot, end-to-end coding agents

Stripe Dot Dev blog · Part 1 and Part 2, February 2026 · long read

<https://stripe.dev/blog/minions-stripes-one-shot-end-to-end-coding-agents>

<https://stripe.dev/blog/minions-stripes-one-shot-end-to-end-coding-agents-part-2>

WHY IT IS ON THIS LIST

Most writing about AI coding is about autocomplete or a chat pane. Stripe's is about a fleet: agents that take a single instruction and come back with a tested pull request, at a volume of more than a thousand merged pull requests a week containing no human-written code, with humans still owning review. It is the clearest public account of what it takes to run coding agents as infrastructure rather than as a tool on one engineer's laptop.

WHAT THEY BUILT

A minion is a task-scoped agent. Work arrives from wherever engineers already are: a reaction on a Slack thread, a bug report, a feature request with enough detail to act on. The minion gets a fresh, isolated devbox, the same kind of environment a human engineer would get, and runs one-shot: no pairing, no back-and-forth. It reads the relevant code, plans, edits, runs the tests, fixes what breaks, and opens a pull request that has already passed CI before a person looks at it.

Two design choices carry the weight. The first is what Stripe calls blueprints: workflows that mix deterministic steps (prepare the environment, run formatters and the test suite, open the pull request in the right shape) with agent loops for the parts that need judgment (understand the task, plan, implement, repair). The deterministic

scaffolding is what makes the output uniform enough to review at volume. The second is the tool plane: the agent core is a fork of Block's open-source Goose, and it reaches Stripe's systems through more than four hundred internal tools exposed over the Model Context Protocol, from code search and the service catalogue to CI and documentation.

The post's central claim is that the model is not the hard part. Minions work because of investments Stripe made for humans long before large language models: hermetic devboxes, a monorepo with consistent conventions, fast and reliable tests, and internal documentation an agent can actually read. Part 2 goes deeper into that Stripe-specific work: making the codebase legible to agents, the verification loops that decide when a change is done, and how the team decides which tasks are minion-shaped.

IDEAS WORTH KEEPING

- **One-shot is a scope decision.** Minions take tasks that can be fully specified up front. Ambiguous work still goes to a person, possibly a person with a pairing tool.
- **Blueprints: determinism where you can, reasoning where you must.** The agent should never improvise steps that never change.
- **The environment is the product.** If the tests need a laptop and a tribal-knowledge setup script, no agent will be trustworthy in it.
- **Tools over one protocol.** Exposing internal systems over MCP turned each of them into something every agent can use without bespoke glue.
- **Measure merge rate and review load,** not lines of code generated.

ASK YOURSELF

Which slice of your backlog is fully specifiable from the ticket alone?
Could an agent run your whole test suite in a clean environment today,
without anyone's laptop?

02

UBER · MARKETPLACE

Running a Software Factory Efficiently at Uber Scale

Uber Engineering blog · 2026 · medium read

<https://www.uber.com/blog/efficient-software-factory/>

WHY IT IS ON THIS LIST

Uber reports that agents now author the majority of its pull requests. This post is about the less glamorous question that follows: how do you keep that affordable and governed when tens of thousands of agent requests a day are hitting hundreds of millions of lines of code and thousands of data tables?

WHAT THEY BUILT

Uber describes an agentic software-development platform with five parts. An AI Context Graph, a unified network of code, services, tables, owners and documents (the post cites tens of millions of nodes and edges across dozens of node and edge types), gives agents organisational knowledge they would otherwise have to search for. An MCP gateway routes every tool call, internal and third-party SaaS alike, through one place, so authentication and policy are enforced centrally; the post says more than a thousand MCP servers sit behind it, each also exposed as a command-line tool that any agent surface can invoke. An LLM gateway abstracts the models. A runtime gives agents somewhere to execute, locally and in the cloud. Governance wraps all of it.

On top of the platform, engineers have built thousands of reusable skills (the post cites more than 3,600, executed tens of thousands of times a day) for migrations, upgrades, documentation and bug fixes. Adoption is

described as near-universal: most of Uber's developers count as agentic coding users, and more than seventy per cent of pull requests are attributed to local or cloud agents.

The efficiency argument is the heart of the piece. Uber's observation is that an ungrounded agent fails slowly rather than cheaply: it keeps sending an ever-larger context window to search one more place. Most agent turns are spent locating information, not generating code, so giving the agent richer information up front, through the context graph and well-shaped skills, is the single biggest lever on both cost and success rate.

IDEAS WORTH KEEPING

- **Grounding is cost control.** Every turn an agent spends searching is tokens you pay and time you wait. The fix is context, not a bigger model.
- **One gateway for tools, one for models.** Policy, authentication, logging and spend live in two places instead of inside every team's agent.
- **Skills are organisational memory.** A procedure captured once becomes something every agent can run.
- **Attribution from day one.** Uber can say what share of pull requests agents produced because it instrumented that before it mattered.

ASK YOURSELF

Where do your agents spend their turns, searching or generating? If a tool call misbehaves tonight, is there one log to read?

Related from Uber: "Solving the Identity Crisis for AI Agents" (2026) on giving agents their own credentials and scopes, and the 2025 posts on Validator and AutoCover, the earlier developer agents.

03

RAMP · FINTECH

Integrations That Write Themselves

Ramp Builders blog · 14 August 2026 · short read

<https://builders.ramp.com/post/integrations-that-write-themselves>

WHY IT IS ON THIS LIST

Every finance or business product has a long tail of integrations that customers ask for and nobody has time to build. Ramp turned that tail into agent-shaped work, and did it twice: once for customers, once for its own engineers.

WHAT THEY BUILT

The customer-facing side: when a customer needs an integration Ramp does not have, an agent researches the third-party API and builds the integration on the spot, within minutes, so the customer can finish what they came to do. The result does not stay trapped in the conversation that created it. It becomes a first-class integration: a step in a procurement workflow, or a tool that the customer's own Ramp agents can call later.

The internal side is an integration factory. An engineer hands it a provider. Ramp's coding agent (Inspect, the internal agent Ramp described building at the start of 2026) reads the provider's API documentation, writes the connector, runs it against test credentials, fixes what fails, and opens a pull request for human review. The post reports seventy-five integrations shipped this way, with work that used to take weeks or months now taking hours.

What makes the piece worth reading is how ordinary the recipe is once you see it: a well-documented external contract, a sandbox to verify against, an automated fix loop, and a human at the pull request. None

of the parts is exotic. The novelty is recognising a class of work with exactly that shape.

IDEAS WORTH KEEPING

- **Look for work specified by someone else's documentation and verifiable with a sandbox.** That is where agents are most trustworthy.
- **Give the agent real feedback, not a mock.** Test credentials and a run-fix loop are what make the output safe to review rather than merely plausible.
- **Land output as a durable artefact,** a connector in the repository or a tool in a registry, not as an answer in a chat.
- **The same harness serves customers and engineers;** what changes is who reviews and what is exposed.

ASK YOURSELF

What is your long tail? Which part of it already has a sandbox and test credentials?

Related from Ramp: "How we built an AI agent that answers 1,000+ data questions a month" (September 2025) and the January 2026 post on building Ramp's own coding agent.

04

ANTHROPIC · DATA & AI

How we built our multi-agent research system

Anthropic Engineering · 13 June 2025 · long read

<https://www.anthropic.com/engineering/multi-agent-research-system>

WHY IT IS ON THIS LIST

The most-cited description of an orchestrator-and-workers agent that actually shipped. It is honest about what worked, what broke, and how much it cost.

WHAT THEY BUILT

Claude's Research feature uses a lead agent that reads the query, plans an approach, and spawns several subagents that search in parallel, each with its own context window, its own tools and a scoped brief. Subagents return condensed findings; the lead synthesises; a final citation step attributes claims to sources. On the team's internal research evaluation, the multi-agent system beat a single agent using the same model by a wide margin, and their analysis found that token usage explained most of the variance in performance: more tokens, spread over more independent contexts, is what buys breadth.

The engineering lessons are the useful part. Prompts had to teach the lead agent how to delegate: each subagent brief needs an objective, an output format, the tools to use and clear boundaries, or subagents duplicate each other or wander. Effort had to be scaled to the query: a fact lookup gets one agent and a few calls; a comparative survey gets many. Tool descriptions turned out to matter as much as prompts; a tool-testing agent that rewrote descriptions after trying the tools cut

task time sharply. Agents were told to start with broad queries and narrow, to use extended thinking as a visible scratchpad, and to make tool calls in parallel, which is where most of the latency win came from.

Evaluation started with a few dozen queries and a model-based judge scoring accuracy, citations, completeness, source quality and tool efficiency, backed by human review for the failures a rubric misses, such as preferring content farms over primary sources. Production brought its own problems: agents are stateful and errors compound, so runs must resume from checkpoints; deployments must not break in-flight agents; every decision needs a trace. The system uses many times the tokens of a chat, so it is reserved for tasks where the value justifies it.

IDEAS WORTH KEEPING

- **Parallelism pays for breadth-first tasks.** It does not pay for tightly coupled work, which is most coding.
- **Delegation is a prompt problem:** objective, output format, tools, boundaries, every time.
- **Match effort to complexity explicitly,** or the agent will over-search simple questions.
- **Tool descriptions are an interface you test,** and the model can help write them.
- **Evaluate end to end,** with a rubric and with humans. Small evaluation sets are enough to start.

ASK YOURSELF

If your agent fails halfway through a long task, can it resume, or does it start again and pay twice?

05

MANUS · DATA & AI

Context Engineering for AI Agents: Lessons from Building Manus

Yichao "Peak" Ji · Manus blog · July 2025 · medium read

<https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>

WHY IT IS ON THIS LIST

The post that made "context engineering" a discipline. It is written by someone whose team rebuilt its agent framework four times, and it is specific about what changed each time.

WHAT THEY LEARNED

Manus chose to build on frontier models and shape their context rather than train models of its own, so that improvements ship in hours instead of weeks. Six lessons follow.

Design around the KV-cache. An agent appends to its context on every step, so input tokens dwarf output tokens, and a cached prefix is an order of magnitude cheaper than an uncached one. Keep the prompt prefix stable (no timestamp at the top), make the context append-only, serialise deterministically, and place cache breakpoints deliberately.

Mask, don't remove. Adding and removing tools mid-task invalidates the cache and confuses the model about tools it saw earlier. Keep the tool set fixed and constrain which tools are allowed at each step by masking during decoding, helped by consistent tool-name prefixes.

Use the file system as context. It is unbounded, persistent and directly operable by the agent. Compress aggressively but restorably: keep the URL, drop the page; keep the path, drop the file.

Manipulate attention through recitation. Manus rewrites a to-do list at the end of the context on every step, pulling the global plan back into the model's recent attention across tasks that average dozens of tool calls.

Keep the wrong stuff in. Failed actions and error traces are evidence. Hiding them makes the model repeat the mistake; leaving them lets it update.

Don't get few-shotted. Uniform, repetitive context makes the model imitate the pattern rather than think. Introduce controlled variation in phrasing and formatting.

IDEAS WORTH KEEPING

- **Cache hit rate is a first-class metric** for any agent that runs more than a few steps.
- **The file system is the cheapest long-term memory you have.**
- **Errors are training signal at inference time.** Error recovery is what separates an agent from a script.
- **Architecture search by trial** (the post calls it "Stochastic Graduate Descent") is normal. Budget for rebuilds.

ASK YOURSELF

What sits at the very top of your system prompt? Does it change between calls?

06

COGNITION · DEVELOPER TOOLS

Multi-Agents: What's Actually Working

Walden Yan · Cognition · 22 April 2026, following "Don't Build Multi-Agents" (12 June 2025) · two short reads

<https://cognition.com/blog/multi-agents-working>

<https://cognition.com/blog/dont-build-multi-agents>

WHY IT IS ON THIS LIST

The two posts bracket a year in which everyone tried agent swarms. The first explained why most of them fail. The second reports what survived.

THE ARGUMENT

"Don't Build Multi-Agents" set out two principles. Share context: every agent must see the full trace of what came before, not a summary of a message. Actions carry implicit decisions: two agents deciding in parallel on different context will make conflicting decisions, and the combined result is incoherent. The example was a game built by parallel subagents that came back with mismatched visual styles. The recommendation was a single-threaded agent, with a dedicated context-compression step for very long tasks. It noted that Claude Code's subagents only answer questions rather than write code, for exactly this reason.

The 2026 follow-up keeps the diagnosis and narrows the prescription. Unstructured swarms of agents negotiating with each other remain, in Cognition's experience, a distraction. What works is a narrower class: several agents contribute intelligence while writes stay single-threaded. The practical shape is map-reduce-and-manage. A manager splits the work, children execute the read-heavy or investigative pieces, and the

manager synthesises and acts. The remaining open problems are communication problems: making a manager-and-children system feel as coherent as one agent on one task.

IDEAS WORTH KEEPING

- **Parallelise reading, not writing.**
- **If two agents can both decide something, one of them is wrong.**
Give each decision a single owner.
- **Context sharing is a hard requirement,** and compression is a model problem worth solving on its own.
- **"Multi-agent" is an architecture decision, not a capability upgrade.** Start single-threaded; add children when a task is provably decomposable.

ASK YOURSELF

In your current design, which agent owns the write? If the answer is "several", what happens when they disagree?

07

CLOUDFLARE · INFRASTRUCTURE

Code Mode: the better way to use MCP

Kenton Varda and Sunil Pai · Cloudflare blog · 26 September 2025, extended in "Code Mode: give agents an entire API in 1,000 tokens" (2026) · medium read

<https://blog.cloudflare.com/code-mode/>

<https://blog.cloudflare.com/code-mode-mcp/>

WHY IT IS ON THIS LIST

A simple observation with large consequences: models are better at writing code that calls an API than at calling that API through tool-calling, because they have read millions of programs and very few synthetic tool-call transcripts.

WHAT THEY BUILT

Instead of exposing an MCP server's tools to the model one by one, Cloudflare's Agents SDK converts the server's schema into a typed TypeScript API with documentation comments. The model gets a single tool: run this TypeScript. The code executes in a sandbox with no network access; the only way out is through bindings that stand for the connected MCP servers, so API keys never enter the model's context, and one script can chain many calls, loop and filter without any intermediate result passing through the model. The sandbox is a Dynamic Worker, a V8 isolate that starts in milliseconds, which is what makes a fresh sandbox per call practical.

The 2026 follow-up scales the idea. With thousands of endpoints, even a TypeScript API is too large to put in context, so the agent gets two tools, search and execute, and discovers the slice of the API it needs on demand. Cloudflare reports its own API surface shrinking from more than a million tokens of tool definitions to roughly a thousand.

Read it alongside Uber's MCP gateway (02) and Stripe's four hundred tools (01): all three are answers to the same question of how an agent reaches a large system without drowning in tool definitions or holding credentials.

IDEAS WORTH KEEPING

- **Tool definitions are context.** The number of tools you can afford is a design constraint, not a detail.
- **Let the model compose.** One script with a loop replaces many round trips and keeps intermediate data out of the context window.
- **Bindings, not credentials.** Sandboxes with capability-style bindings give an agent power without giving it secrets.
- **Discovery over enumeration:** search the API, then execute.

ASK YOURSELF

How many tokens do your tool definitions cost on every single turn?
Could the model write the glue instead?

Related from Cloudflare: "Sandboxing AI agents, 100x faster" (2026), on the Dynamic Workers that make per-call isolates cheap.

08

UBER · MARKETPLACE

QueryGPT: Natural Language to SQL Using Generative AI

Uber Engineering blog · 19 September 2024 · medium read

<https://www.uber.com/blog/query-gpt/>

WHY IT IS ON THIS LIST

Still a model of how to take a demo, in this case a hackathon text-to-SQL prototype, to a tool that operations teams use every day. It is candid about what did not work.

WHAT THEY BUILT

Uber's data platform serves on the order of a million interactive queries a month, many written by operations and support staff who are not SQL experts. QueryGPT takes an English question and produces a query against the warehouse. The first version, from an internal hackathon in 2023, retrieved sample queries and schemas by vector similarity and stuffed them into one prompt. It hit two walls: schemas at Uber run to hundreds of columns, so a handful of tables exhausts the context window, and similarity search across the whole warehouse kept picking the wrong tables.

The production design is a pipeline of small, specialised steps. Workspaces are curated collections of tables and sample queries for a business domain (Mobility, Ads, Core Services and so on); users can also build their own. An intent step maps the question to workspaces. A table step picks tables within them, and the user can confirm or correct the choice, a cheap human checkpoint that fixed the worst failures. A column-pruning step drops irrelevant columns from the schemas so the generation prompt fits. Only then does the generation step write the SQL, with few-shot samples drawn from the workspace.

Evaluation uses a fixed set of golden questions with per-stage metrics (intent accuracy, table overlap, whether the query runs, whether it returns rows) plus qualitative review of query quality. At the time of writing, the limited release had a few hundred daily users, most of whom said it saved them time over writing the query from scratch.

IDEAS WORTH KEEPING

- **Decompose.** A chain of narrow model calls with checkable outputs beats one clever prompt.
- **Curate the retrieval corpus.** The workspaces are the product; the model is interchangeable.
- **Insert cheap human confirmations** where the model is least reliable.
- **Measure each stage.** End-to-end accuracy hides where the errors come from.

ASK YOURSELF

Where is the one confirmation you could ask a user for that would remove most of your agent's worst mistakes?

09

AIRBNB · MARKETPLACE

Automation Platform v2: Improving conversational AI at Airbnb

Chutian (Jeremy) Wang · The Airbnb Tech Blog · October 2024 · medium read

<https://medium.com/airbnb-engineering/automation-platform-v2-improving-conversational-ai-at-airbnb-d86c9386e0cb>

WHY IT IS ON THIS LIST

A production platform for LLM-driven customer support that deliberately does not go all-in on the model. It shows how to combine deterministic workflows with model reasoning in one system, and which supporting components you need before you can trust it with customers.

WHAT THEY BUILT

Version 1 of Airbnb's Automation Platform ran conversational flows as static, predefined workflows: robust, but rigid. Version 2 adds LLM-powered capabilities and keeps the workflows, choosing per use case. The LLM path is a chain-of-thought workflow: the model reasons about what to do next, selects a tool (check a reservation, look up availability, read a policy), sees the result, and continues until it can respond. Tools are the contract between the model and Airbnb's systems.

Around the loop sit the components that make it operable. A context manager gathers what the model needs (trip details, prior messages, retrieved policy text) from declared sources and ranks it into the prompt. A guardrails framework runs checks in parallel with the response (relevance, safety, policy) so unsafe or off-topic output is caught before it reaches the customer. An observability layer records prompts, tool calls, tokens and latency for every turn, which is what makes a conversation nobody watched debuggable afterwards. Airbnb's later

posts describe feedback from human support agents flowing back into the system and the share of issues resolved without a person continuing to rise.

IDEAS WORTH KEEPING

- **Hybrid by design.** Keep deterministic workflows where the answer must be exact; use the model where flexibility helps.
- **Context management is its own component** with declared inputs, not string concatenation inside a prompt.
- **Guardrails in parallel, not in series,** keep latency flat.
- **Trace every prompt and tool call.** Support conversations are the ones you will need to replay.

ASK YOURSELF

Which of your flows must be exact every time, and which only need to be helpful? Are they built the same way?

Related from Airbnb: the 2025 posts on the agent-in-the-loop feedback framework for the customer-support system, and "Accelerating large-scale test migration with LLMs" (March 2025).

10

LINKEDIN · SOCIAL

How we engineered LinkedIn's Hiring Assistant

LinkedIn Engineering blog · 21 October 2025, with a June 2026 follow-up on international markets · medium read

<https://www.linkedin.com/blog/engineering/ai/how-we-engineered-linkedins-hiring-assistant>

<https://www.linkedin.com/blog/engineering/ai/engineering-linkedin-hiring-assistant-for-international-markets>

WHY IT IS ON THIS LIST

LinkedIn's first production agent, and a rare description of the platform under a consumer-scale agent: not one clever loop, but orchestration, memory, a skills registry and a client SDK.

WHAT THEY BUILT

Hiring Assistant helps recruiters take a role from intake to outreach. As described, a supervisor agent coordinates specialised sub-agents for intake (understanding the role), sourcing (searching for candidates), evaluation (screening against the brief) and outreach (messaging). They run on LinkedIn's agent platform: asynchronous, message-based orchestration so long-running tasks survive failures; a skills registry through which agents discover tools dynamically; centralised prompt management and an inference abstraction over models; and layered memory, including an experiential memory that learns a recruiter's preferences from their behaviour, backed by a preference model trained on recruiter actions.

On the client side, an SDK embeds the assistant into the recruiter's existing workflow with interfaces that adapt to what the agent needs at each step, across chat, voice and typing assistance. The recruiter remains the decision-maker: the agent surfaces and recommends; the

person contacts and hires. LinkedIn's business announcements claim hours saved per role and far fewer profiles reviewed, and the 2026 follow-up covers what changed when the same agent had to work across languages and markets.

IDEAS WORTH KEEPING

- **Agents at scale are asynchronous systems.** Messaging, retries and resumability come before prompts.
- **Memory is layered:** task state, session context, and long-term preferences learned from behaviour.
- **A skills registry** lets teams add capabilities without redeploying the agent.
- **The interface must adapt to the agent** (approve, correct, confirm), not just show a chat.

ASK YOURSELF

What does your agent remember about a user between sessions, and who can inspect or reset it?

SYNTHESIS

Patterns across the ten

Read together, the posts agree on more than their authors might expect. Seven ideas recur, in different words, across companies that never compared notes.

PATTERN	WHAT IT MEANS IN PRACTICE	SEEN IN
Writes stay single-threaded	Parallelise research and reading; give every decision one owner; synthesise in one place.	06 · 04 · 01
Context is the product	KV-cache discipline, file systems as memory, context graphs, curated workspaces. Most agent failures are context failures.	05 · 02 · 08 · 04
Tools want a gateway, or want to be code	One protocol (MCP), one gateway for auth and policy, or a typed API the model programs against.	07 · 02 · 01
Deterministic scaffolding around stochastic loops	Blueprints, hybrid workflows, staged pipelines. The model reasons; the harness sequences.	01 · 09 · 08
Verification is the trust mechanism	Tests in a clean environment, real credentials in a sandbox, guardrails, rubric-based judges, and a human at the pull request.	03 · 01 · 09 · 04
Cost is an architecture problem	Grounding to cut search turns, cache-stable prompts, effort budgets per task class, fewer tool definitions per turn.	02 · 05 · 04 · 07

PATTERN	WHAT IT MEANS IN PRACTICE	SEEN IN
Ship the platform, not the agent	Gateways, skills registries, prompt management, memory layers and tracing outlive any single agent.	02 · 10 · 09

FURTHER READING

If the ten were not enough

Eight more posts, all free to read, that the ten above cite or argue with.

Building effective agents · Anthropic · December 2024

The workflows-versus-agents taxonomy most of the field now uses: prompt chaining, routing, parallelisation, orchestrator-workers, evaluator-optimiser.

<https://www.anthropic.com/engineering/building-effective-agents>

Effective context engineering for AI agents · Anthropic · September 2025

The attention budget, compaction, structured note-taking and sub-agent architectures, as a companion to the Manus post.

<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

Writing tools for agents · Anthropic · September 2025

How to design, describe and evaluate tools, with the model helping to improve its own tool descriptions.

<https://www.anthropic.com/engineering/writing-tools-for-agents>

A practical guide to building agents · OpenAI · April 2025

A short guide to when an agent is warranted, single versus multi-agent systems, and guardrails.

<https://openai.com/business/guides-and-resources/a-practical-guide-to-building-ai-agents/>

GitHub Copilot coding agent 101 · GitHub · 2025

The mainstream version of the Stripe idea: an agent that runs in GitHub Actions, picks up an issue, and opens a pull request.

<https://github.blog/ai-and-ml/github-copilot/github-copilot-coding-agent-101-getting-started-with-agentic-workflows-on-github/>

Introducing codename goose · Block · January 2025

The open-source, MCP-native agent that Stripe forked for its minions.

<https://block.xyz/inside/block-open-source-introduces-codename-goose>

Sandboxing AI agents, 100x faster · Cloudflare · 2026

The Dynamic Workers underneath Code Mode: isolates cheap enough to give every tool call its own sandbox.

<https://blog.cloudflare.com/dynamic-workers/>

Solving the Identity Crisis for AI Agents · Uber · 2026

Agents need their own identities, scopes and audit trails, not a borrowed human token.

<https://www.uber.com/blog/solving-the-agent-identity-crisis/>

ABOUT THIS PACK

How it was made, and how to use it

The digests are original summaries written from the posts themselves. They paraphrase; the few short phrases quoted are marked as such. No article text is reproduced, and nothing here replaces the originals, which are all free to read at the links printed under each title. Where a post reports figures, the digest repeats them as that post's claims rather than as independent facts.

The same ten posts, and the wider stream they come from, are indexed under the Agentic AI topic at drupesh2008.github.io/tech-blogs, which refreshes itself every six hours from each publisher's own feed and links straight back to the source.

Suggested use: read one thread per day, keep the "ask yourself" questions for the notebook, and when you are back online open the originals for the diagrams and code that a digest cannot carry.

PREPARED 29 SEPTEMBER 2026 · SET IN INSTRUMENT SERIF, INTER AND
JETBRAINS MONO · A5 FOR PHONES AND TABLETS